CODE APEX 3.0 • iMOCHA TRACK

AI Hackathon Problem Statements

Five problem statements from iMocha, led by Vibe Coding, our AI-era coding assessment.

Problem statements

IM-01 Vibe Coding: The AI-Era Coding Assessment
IM-02 Autonomous AI QA Engineer
IM-03 Skill Passport: Verified Skills from Real Work
IM-04 AI Job Simulator: Practise Real Work Scenarios
IM-05 Codebase Mentor: AI Onboarding for Developers

Prepared for the Department of Computer Science & Engineering (Artificial Intelligence),
Vishwakarma Institute of Technology, Pune

INTRODUCTION

About the iMocha track

iMocha is an AI-powered skills intelligence company that helps organisations assess, understand and grow the skills of their people. For CODE APEX 3.0, iMocha is contributing a dedicated track of five problem statements across AI and Generative AI, AI-enabled assessments, skill intelligence, automation, software engineering and QA.

The first problem statement, **Vibe Coding**, comes directly from a product iMocha is prototyping today. The other four focus on real-world problems where AI agents can complete meaningful work and back their results with evidence.

Problem statements at a glance

ID	Problem statement	Focus area
IM-01	Vibe Coding: The AI-Era Coding Assessment	AI-enabled assessments • Generative AI
IM-02	Autonomous AI QA Engineer	QA • Automation • Software engineering
IM-03	Skill Passport: Verified Skills from Real Work	Skill intelligence • Generative AI
IM-04	AI Job Simulator: Practise Real Work Scenarios	AI & Generative AI • AI-enabled assessments
IM-05	Codebase Mentor: AI Onboarding for Developers	Software engineering • Generative AI

How each problem statement is structured

- **Background** and **the challenge**: the real problem and why it matters.
- **Expected solution** and **expected outcomes**: what a strong submission should do and show.
- **Stage-wise expectations**: how the same idea should grow across the three stages of the hackathon.

> **One idea, three stages.** Teams choose one problem statement and carry the same idea through Stage 1 (idea and presentation), Stage 2 (online prototype) and Stage 3 (the 24-hour offline grand finale), making it more complete at each stage.

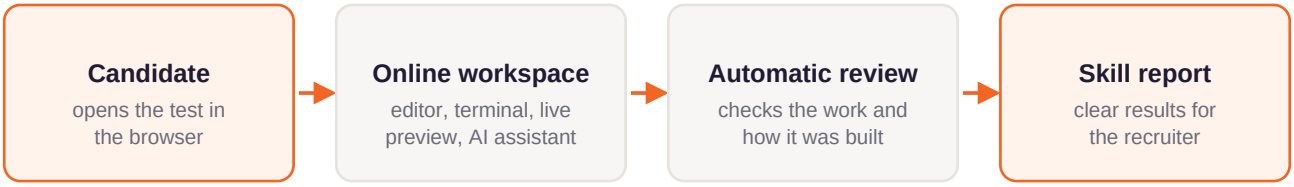
PROBLEM STATEMENT IM-01

Vibe Coding: The AI-Era Coding Assessment

Focus area	Category	Who it is for
AI-enabled assessments • Generative AI	Software	Candidates (students and job seekers) and recruiters

Background

Developers today build software together with AI: they plan, prompt an assistant, check what it produces and fix its mistakes. Most coding tests still ask candidates to solve puzzles alone, without AI, against a timer. iMocha is prototyping **Vibe Coding**, an assessment where the candidate gets a small real-world task and a complete coding workspace in the browser with an AI assistant, builds something that actually works, and the recruiter receives a report on the skills shown. The basics work, but the candidate experience and the skill report are still open questions.



How a Vibe Coding assessment works today. The highlighted ends (the candidate and the report) are where we want ideas.

The challenge

> Design and build a prototype that makes the Vibe Coding experience better for **candidates**, or makes the **skill report** more useful, fair and easy to understand, or both.

Expected solution

- **Candidate experience:** a welcoming onboarding or practice mode, clear task guidance, and an AI assistant that coaches and explains rather than only handing over code.
- **Showing how the candidate worked:** capture the journey (planning, prompting, testing, fixing) in a way candidates find fair and transparent.
- **A clear skill report:** a one-page visual summary of skills, strengths and gaps, with each result explained in plain language and backed by examples from the candidate's own work.
- **AI collaboration insights:** how clearly the candidate stated intent, whether they checked the AI's output, and whether they understood the code it produced.
- **Feedback for candidates:** a personal summary of what went well and what to practise next, without revealing the hidden checks.

Expected outcomes

- A working prototype; a mock workspace or a recorded sample session is a perfectly good starting point.
- A sample skill report generated from at least one candidate session.

- A clear explanation of why the report is fair and easy for a recruiter to trust.

Stage-wise expectations

The same idea is carried through all three stages and grows at each one.

Stage	What we expect to see
Stage 1 **Idea & presentation**	The problem from the candidate's or recruiter's point of view, the chosen focus (experience, report or both), the solution design, and mock-ups of key screens or a sample report.
Stage 2 **Online prototype**	A working core flow on a sample task: a candidate solves a small task with an AI assistant, the session is captured, and a first report or feedback summary is generated.
Stage 3 **24-hour grand finale**	A complete end-to-end experience: richer insights, a polished report, a live demo with a real session, and how fairness has been considered.

PROBLEM STATEMENT IM-02

Autonomous AI QA Engineer

Focus area	Category	Who it is for
QA • Automation • Software engineering	Software	Developers and QA teams, especially in small teams with little testing capacity

Background

Software ships faster than it can be tested. Manual testing is slow and repetitive, automated test suites fall behind new features, and bug reports often arrive without clear steps to reproduce them. Small teams frequently have no dedicated tester at all.

The challenge

> Build an AI agent that tests a web application the way a careful human tester would: it explores the app, decides what to test, runs the tests, and reports real bugs with evidence.

Expected solution

- Given a web app address and a short description of what it should do, the agent explores the app like a real user.
- It generates test cases for the main user journeys and edge cases, and runs them in a browser.
- It files clear bug reports: steps to reproduce, expected versus actual behaviour, and screenshots.
- It turns successful journeys into reusable automated tests that the team can keep running.
- Optional: several cooperating agents, for example an explorer, a tester and a reporter.

Expected outcomes

- On a sample app with deliberately planted bugs, the agent finds and correctly reports a measurable share of them.
- Generated tests run on their own and can be repeated.
- Every bug report can be reproduced by a person following its steps.

Stage-wise expectations

The same idea is carried through all three stages and grows at each one.

Stage	What we expect to see
Stage 1 **Idea & presentation**	The testing problem being solved, the agent's design (what it sees, decides and does), the target sample app, and how accuracy and coverage will be measured.
Stage 2 **Online prototype**	The agent tests one key journey (for example sign-up and login) on a sample app, produces a test, and files at least one evidence-backed bug report.
Stage 3 **24-hour grand finale**	Exploration across several journeys, cooperating agents, results on a planted-bug benchmark, and a simple dashboard of findings.

PROBLEM STATEMENT IM-03

Skill Passport: Verified Skills from Real Work

Focus area	Category	Who it is for
Skill intelligence • Generative AI	Software	Students and job seekers, recruiters and college placement cells

Background

Resumes list skills but rarely prove them. A student's real abilities are spread across code repositories, project reports, portfolios and certificates. Recruiters cannot check all of this quickly, and learners often do not know which skills they are missing for the role they want.

The challenge

> Build an AI system that turns a person's real work into a verified skill profile, where every skill is backed by evidence and linked to a clear path for growth.

Expected solution

- Analyse a person's work, such as code repositories, project documents, a portfolio or certificates.
- Produce a skill profile in which every skill and level is linked to cited evidence (a project, a file or a piece of work).
- Compare the profile with a target role and show matches and gaps.
- Generate a personalised learning plan for closing the gaps.
- Offer a shareable "skill passport" view that a recruiter can read in under a minute.
- Guard against invented claims: a skill without evidence should never appear as verified.

Expected outcomes

- Skill passports for a few sample users built from public or sample data.
- Every claimed skill traceable to its evidence.
- A clear explanation of how well a person matches a given role, and why.

Stage-wise expectations

The same idea is carried through all three stages and grows at each one.

Stage	What we expect to see
Stage 1 **Idea & presentation**	The skill framework to be used, the data sources, how evidence will be linked to each skill, and mock-ups of the passport.
Stage 2 **Online prototype**	A passport generated from one public code repository and a resume, with every skill linked to evidence.
Stage 3 **24-hour grand finale**	Multiple data sources, role matching, learning plans, a recruiter view, and checks that prevent unsupported claims.

PROBLEM STATEMENT IM-04

AI Job Simulator: Practise Real Work Scenarios

Focus area	Category	Who it is for
AI & Generative AI • AI-enabled assessments	Software	Students preparing for their first job, and employers who want realistic evaluation

Background

Interviews and tests rarely show how someone handles real work: a production incident, a code review, an unclear requirement from a client, or a difficult conversation with a teammate. Students seldom get to practise these situations before their first job.

The challenge

> Build an AI-powered simulator in which people practise realistic workplace scenarios with AI characters and receive fair, evidence-based feedback afterwards.

Expected solution

- Realistic scenarios with AI characters such as a manager, a client or a teammate, through chat or voice.
- Scenarios that react to what the person says and does, rather than following a fixed script.
- A debrief at the end, graded against a clear rubric, that quotes moments from the conversation as evidence.
- Optional: generate new scenarios from a job description, and track progress over several attempts.

Expected outcomes

- At least two or three complete, realistic scenarios.
- A rubric-based debrief in which every point is backed by a moment from the session.
- Feedback that a student would find genuinely useful for their next attempt.

Stage-wise expectations

The same idea is carried through all three stages and grows at each one.

Stage	What we expect to see
Stage 1 **Idea & presentation**	The roles and scenarios chosen, the rubric, the conversation design, and how feedback will be made fair and specific.
Stage 2 **Online prototype**	One complete scenario in text, from start to debrief, with rubric-based feedback.
Stage 3 **24-hour grand finale**	Voice or video interaction, several scenarios, scenario generation from a job description, and progress tracking.

PROBLEM STATEMENT IM-05

Codebase Mentor: AI Onboarding for Developers

Focus area	Category	Who it is for
Software engineering • Generative AI	Software	Interns and new developers, open-source contributors, and engineering teams

Background

New developers often spend weeks understanding an unfamiliar codebase. Much of the important knowledge lives only in senior engineers' heads and is lost when they move on, while written documentation is usually incomplete or out of date.

The challenge

> Build an AI mentor that reads a code repository and helps a newcomer understand it quickly, with every answer grounded in the actual code.

Expected solution

- An architecture overview: the main parts of the system and how they connect.
- Guided tours of key flows, for example "what happens when a user signs up".
- Answers to "how" and "why" questions that link to the exact files and lines they are based on.
- Suggested starter tasks that help a newcomer make a first contribution.
- Optional: keep the generated documentation up to date as the code changes.

Expected outcomes

- Works on a real open-source repository.
- Every answer cites the code it relies on, so it can be checked.
- A small test with peers showing that newcomers understand the codebase faster with the mentor.

Stage-wise expectations

The same idea is carried through all three stages and grows at each one.

Stage	What we expect to see
Stage 1 **Idea & presentation**	The onboarding problem, the target users, the approach, and how answers will stay grounded in the code.
Stage 2 **Online prototype**	Question answering with cited code, and an architecture overview, for one repository.
Stage 3 **24-hour grand finale**	Guided tours, starter tasks, documentation that updates with code changes, and results from a small user test.

FOR ALL PROBLEM STATEMENTS

General guidelines

- **Any technology stack** may be used. Prototypes can be standalone; no access to iMocha systems is needed.
- **Use public, sample or synthetic data only.** Do not use real people's personal information without consent.
- **Use your own accounts and API keys** for any AI or cloud service, and never publish keys in your code.
- **AI tools are welcome** while building. Please state which tools and models were used.
- **Credit what you reuse.** Open-source libraries, datasets and references should be acknowledged.
- **Be original.** Solutions should be the team's own work.